Durham University

Applied security (COMP3761)
Fuzzing and software testing

Prof David Oswald & Dr Charles Morisset

david.f.oswald@durham.ac.uk

MCS1033

Adapted from slides by Dr Marius Muench – thanks!

Durham University

Before we begin:
Stack overflow walkthrough

Week	Lecture topic	Practical topic	Further reading and material
Week 12 (5 Oct)	Intro & memory safety (recap) Slides Recording	No workshop in first week	Smashing the stack for fun and profit A different lab for ret2libc ROP papers and talks from the inventors Awesome Reverse Engineering
Week 13 (12 Oct)	Memory safety recap, fuzzing & software testing part 1	Binary exploitation exercise part 1	The fuzzing book The LibAFL book Awesome Fuzzing on Github
Week 14 (19 Oct)	ROP recap, fuzzing & software testing part 2	Binary exploitation exercise part 2, fuzzing exercise	Autopsy Forensics Tool Awesome Forensics on Github The Computer Forensic Reference DataSet Portal
Week 15 (26 Oct)	(Mobile) Forensics - tools, techniques, pitfalls, regulations	Fuzzing exercise part 2	
Week 16 (2 Nov)	IoT & embedded systems (“hardware testing”)	“The Pi exercise” (details to be announced)	
Week 17 (9 Nov)	Hardware side channels	Forensics OR IoT/hardware attacks exercise	

Plan for today/next week

- Understand the basics of software testing
- Understand important testing metrics
- Dive into fuzzing as one of the most popular security testing methods

"Testing is the process of analyzing a software item to detect the differences between existing and required conditions (that is, bugs) and to evaluate the features of the software item."

ANSI/IEEE 1059-1993 (withdrawn)

Why Software Testing is important

High-level goal: Ensure software behaves as intended

- Software developers are humans
  - We all make mistakes
  - These mistakes can lead to faults/bugs in software

- Testing helps us to identify and fix bugs!
  - Preventing a system from failure
  - Includes identification of security critical bugs (i.e., vulnerabilities)

The four levels of testing



Unit Testing

Target: Individual components



Integration Testing

Target: Modules in combination



System Testing

Target: The fully integrated system



Acceptance Testing

Target: Users of the system

- Tests functionality of basic building blocks (e.g, single functions)
- Focuses on components in isolation
- May include stubs or drivers to execute test

- Example:

```
def add(a, b):
    return a + b

def test():
    assert add(1,1) == 2
```

Durham University

The four levels of testing



Unit Testing
Target: Individual components



Integration Testing
Target: Modules in combination



System Testing
Target: The fully integrated system



Acceptance Testing
Target: Users of the system

- Tests functionality of building blocks when combined together
- Focuses on interaction between components
- Usually groups (unit tested) modules together following fixed strategies:
  - E.g, top-down, bottom-up, hybrid

- Example:

```
def test():
    assert sub(add(1,1),2) == 0
```

Durham University

The four levels of testing



Unit Testing
Target: Individual components



Integration Testing
Target: Modules in combination



System Testing
Target: The fully integrated system



Acceptance Testing
Target: Users of the system

- Tests functionality of fully integrated system
- Focuses on interaction between all components and the correctness of system as a whole
- May be guided by requirement specifications

- Example:

```
$ calculator.py "2+2-1"
```

Durham University

The four levels of testing



Unit Testing
Target: Individual components



Integration Testing
Target: Modules in combination



System Testing
Target: The fully integrated system

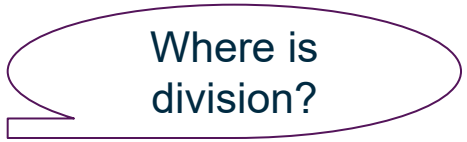


Acceptance Testing
Target: Users of the system

- Tests whether system fulfills acceptance criteria (e.g., requirement/specification)
- May involve (end-)users of the system
- Includes well-known testing types:
  - Alpha testing, beta testing

- Example:






```
$ calculator.py 2+2/1
ERROR: No such operand "/"
```

Durham University

The “Box Model”

Testing can be classified based on the information provided to the test/tester:



Black-box

- No information about the internal structure
- Tester observes input and output



Grey-box

- Partial information about the internal structure
- Tester can create informed tests



White-box

- Full information about the internal structure
- Tester can leverage source code

Static vs. Dynamic Testing

Static

- Also called “off-line” testing
- Testing without execution
- *Reasons* about the code
- May create false-positives
- Can be aided by LLMs now

Dynamic

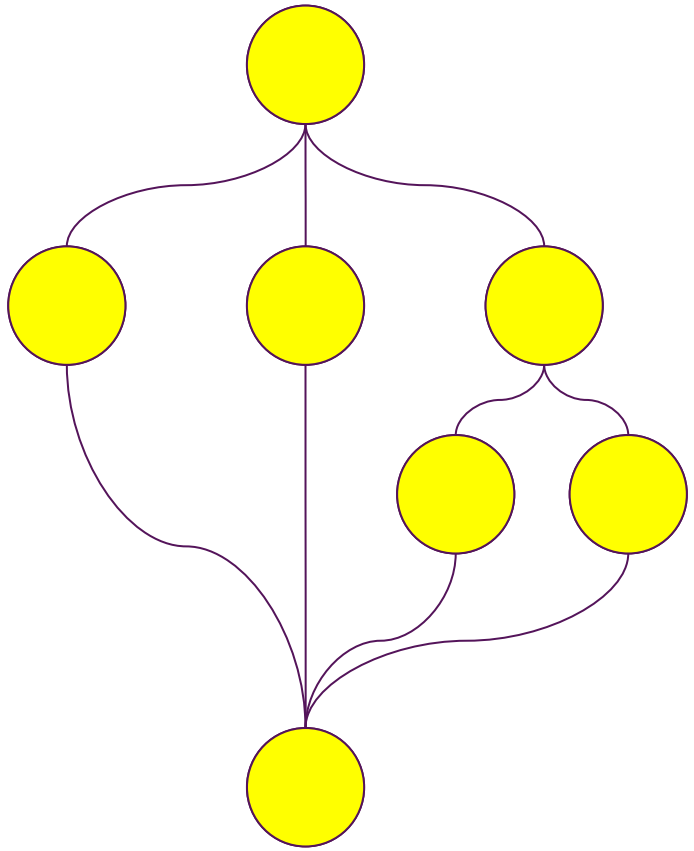
- Also called “on-line” testing
- Testing by executing the target
- Observes the code’s *behavior*
- May not cover full program

Test Coverage

- Metrics to understand how complete given tests are
  - Provides insights which parts of a software are tested
- Multiple types, e.g.:
  - Requirement coverage (*which of the requirements are tested?)*
  - Compatibility coverage (*which hardware/software targets is the target tested with?*)
  - Code coverage (*which parts of the code are tested?*)
- We will look more into code coverage
  - Useful metric for security testing
  - More code coverage == more parts of the system tested

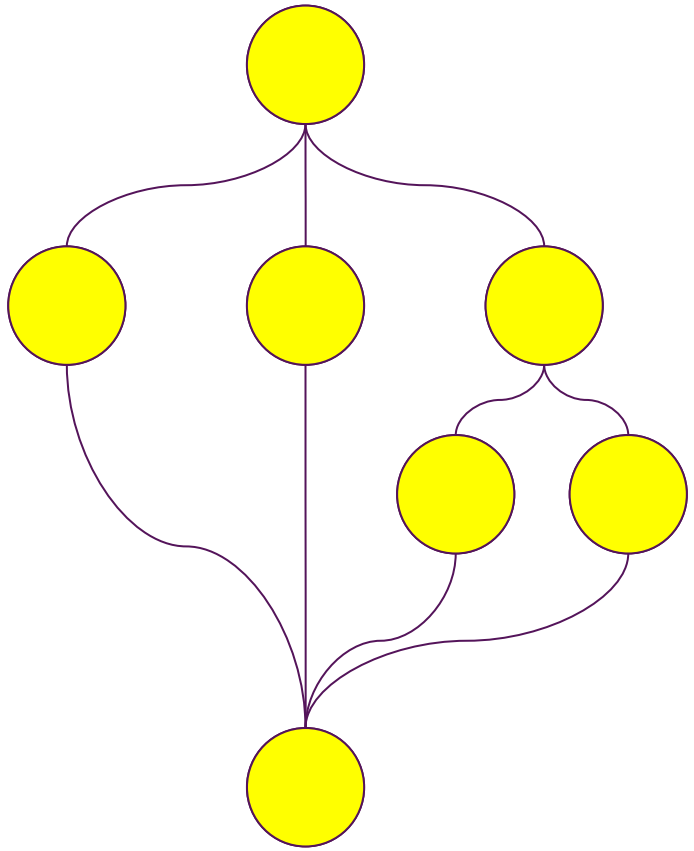
Code Coverage: Ideal scenario

```
void my_function(char in1, char in2) {
    switch (in1) {
    case ‘A’: handle_A(); break;
    case ‘B’:
        if (in2 == ‘S’) {
            handle_B_special(); break;
        } else {
            handle_B_default(); break;
        }
    default: housekeeping();
    }
    return;
}
```



Code Coverage Types

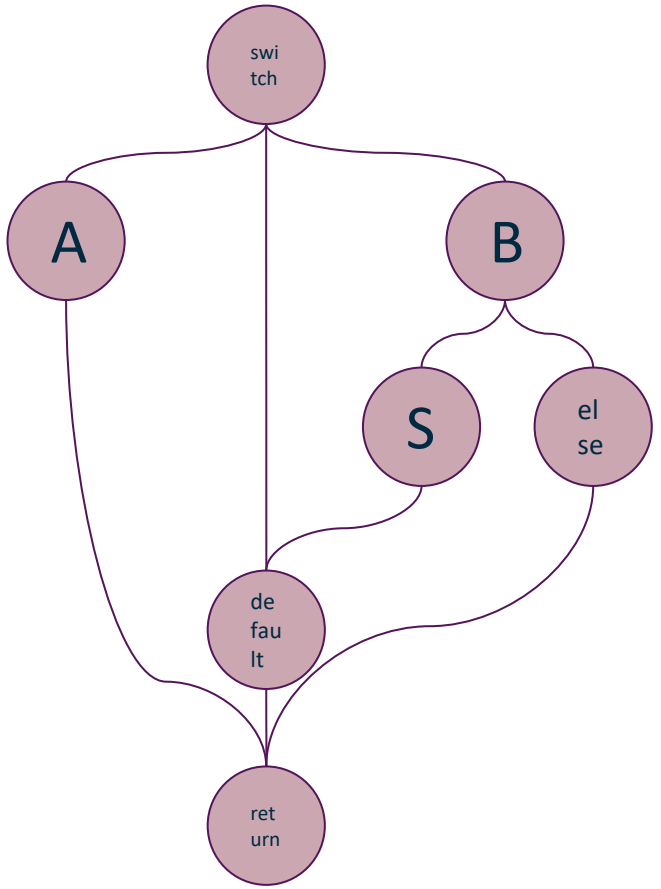
- Function coverage
  - Which parts (functions) of the *software* are executed?
- Line coverage
  - Which lines of the *source code* are executed?
- Block coverage
  - Which parts of the *machine code* (basic blocks i.e. control flow graph nodes) are executed?
- Edge coverage
  - Which transitions of the control flow graph (i.e. edges) are executed?



Quiz

```
void my_function(char in1, char in2) {
   switch (in1) {
   case ‘A’: handle_A(); break;
   case ‘B’:
       if (in2 == ‘S’) {
           handle_B_special();
       } else {
           handle_B_default(); break;
       }
   default: housekeeping();
   }
   return;
}
```

No break!



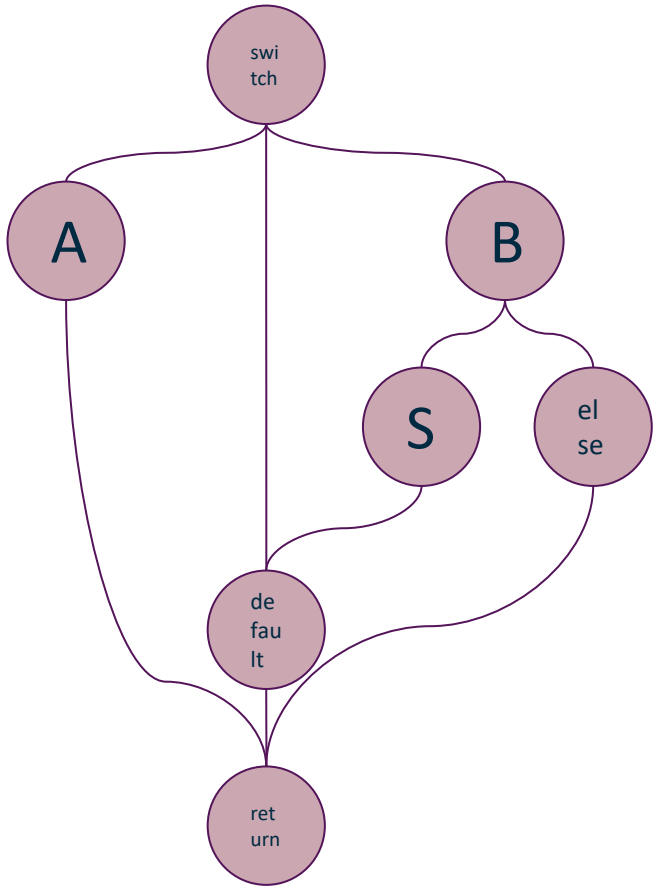
How many test cases (i.e., combinations of inputs) are required to achieve full:

1) Function coverage?
1) Line coverage?
1) Block coverage?
1) Edge coverage?

Durham University

Quiz

```
void my_function(char in1, char in2) {
   switch (in1) {
   case ‘A’: handle_A(); break;
   case ‘B’:
       if (in2 == ‘S’) {
           handle_B_special();
       } else {
           handle_B_default(); break;
       }
   default: housekeeping();
   }
   return;
}
```



How many test cases (i.e., combinations of inputs) are required to achieve full:

1) Function coverage? 3
1) Line coverage? 3
1) Block coverage? 3
1) Edge coverage? 4

Example test cases: (1) AX, BX, BS – (2) AX, BX, BS – (3) AX, BX, BS – (4) AX, BX, BS, XX

Static Code Analysis

- Most common scenario: source code analysis
  - However, techniques for static binary code analysis exist as well!
- Example: Detecting vulnerable calls to APIs
  - Could be as simple as `grep strcpy`
  - Or using advanced tooling such as CodeQL [1]
- Scales well to full programs
  - But may report many false positives

```
import python

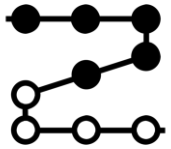
from Function f
where count(f.getAnArg()) > 7
     select f
```

CodeQL Example [2]

[1] https://codeql.github.com/
[2] https://codeql.github.com/docs/writing-codeql-queries/introduction-to-ql/

Taint Analysis

- Testing techniques that can be implemented both dynamically or statically
- Aims to identify flow of user input throughout the program (and whether it has security implications)
- Core idea:
  - Tagging user input and tracking (“tainting”) use through the program
  - Check whether tainted values ever reach vulnerable functions (“sinks”) without prior sanitization

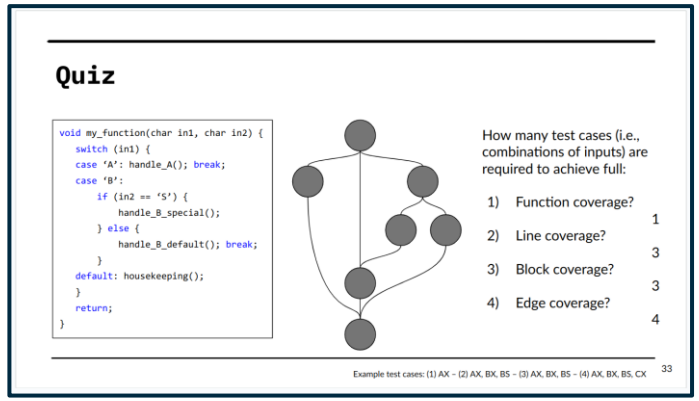


Symbolic Execution

- Instead of testing/executing the target with *concrete* inputs “execute” it with *symbolic* values
  - Goal: “Exploring” all states of a program

- On each branch, collect constraints on inputs
  - If test case to reach a certain state is required, invoke a solver

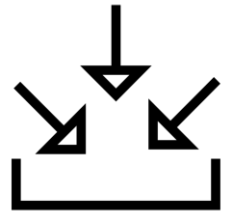
- Scalability issues:
  - State/Path explosion
  - Constraint solving



Simplified example: interpreting inputs to program state relations

Fuzzing

- Executing the target program repeatedly with (pseudo-)random input
  - Mutate existing or generate new input for each run
- Monitor program behaviour for crashes
  - Crashes demonstrate shortcomings (failures) of the target software
- Scales well to full programs
  - But may miss substantial parts of the target



Fuzzing: Origin

1988: Class project introduced by Bart Miller at University of Wisconsin-Madison:

- Goal: test reliability of UNIX command line programs
- Idea: Feeding random inputs to programs and check for crashes
- First time usage of the term “fuzz” to describe testing approach
- Crashed up to 25-33% of programs (!)
- Results subsequently published in 1990

Original fuzzing assignment: https://pages.cs.wisc.edu/~bart/fuzz/CS736-Projects-f1988.pdf
Original paper: https://www.paradyn.org/papers/fuzz.pdf

Fuzzing: 30 years later

- One of the core automated security testing technique!
  - Incredible amount of fuzzers
  - Wide adoption in the industry
  - Large body of academic work

- Core principles of classic fuzzing still intact, however:
  - Smarter ways of input construction
  - Potentially more knowledge about the target

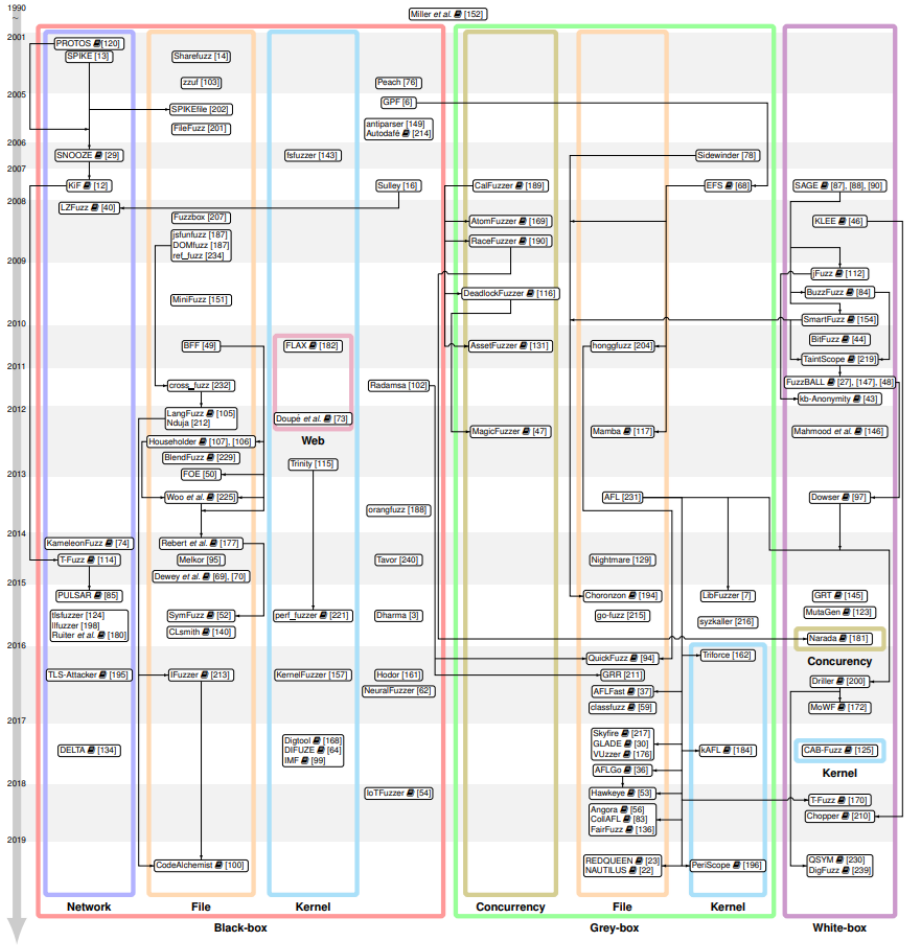


Image Source: Manès, Valentin Jean Marie, et al. "The art, science, and engineering of fuzzing: A survey." IEEE Transactions on Software Engineering'19

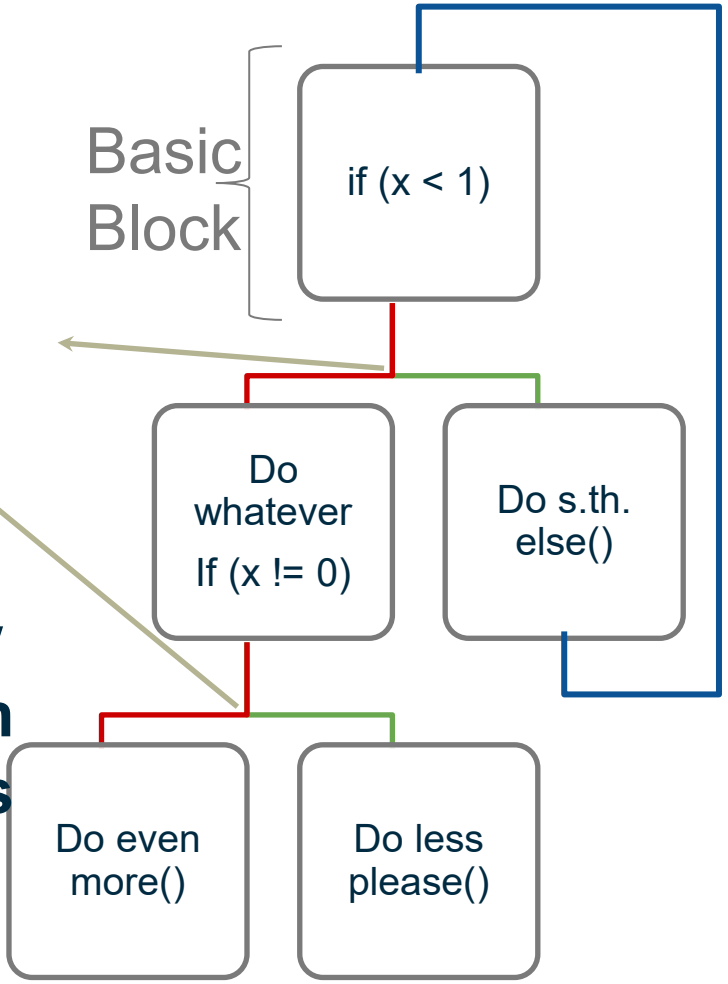
Feedback-driven Fuzzing

- Core idea of modern fuzzing is using ***feedback*** from individual fuzzing runs to guide test case creation

- Feedback examples:
  - Code coverage
  - Response times
  - State coverage

- Requires instrumentation of target or execution environment
  - May induce performance penalty over plain black-box fuzzing
  - May allow to explore target faster → acceptable trade-off

Feedback Example: Coverage-Guided Fuzzing



Count how often these happen.

↳**Multiple basic blocks in a row describe a Path**

↳**Favor new Paths**

- Most common form of feedback-driven fuzzing
- Code coverage as feedback
- Fuzzer keeps track of reached path and favors new paths

Examples:

AFLplusplus, LibAFL, Hongfuzz, LLVM, libFuzzer, Jackalope, Syzkaller

Durham University

Instrumentation?

“Instrumenting” a program means to change its functionality

This requires the additions/modifications to the code

Typical ways of instrumentation:

- Source-based
- Compile-time based
- Binary-based

```
; int __fastcall main(int argc, const char **argv, const char **envp
public main
main proc near

var_A0= qword ptr -0A0h
var_94= dword ptr -94h
s= byte ptr -90h
var_8= qword ptr -8

; __unwind {
endbr64
push    rbp
mov     rbp, rsp
sub     rsp, 0A0h
mov     [rbp+var_94], edi
mov     [rbp+var_A0], rsi
mov     rax, fs:28h
mov     [rbp+var_8], rax
xor     eax, eax
```



```
; int __fastcall main(int argc, const char **argv, const char **envp)
public main
main proc near

var_D8= qword ptr -0D8h
var_D0= qword ptr -0D0h
line= byte ptr -0C8h
var_40= qword ptr -40h

argc = rdi              ; int
argv = rsi              ; char **
; __unwind {
endbr64
push    r15
push    r14
push    r13
push    r12
push    rbp
push    rbx
sub     rsp, 0A8h
lea     r13, __afl_area_ptr
mov     r15, cs:off_AFD8
mov     rax, fs:28h
mov     [rsp+0D8h+var_40], rax
xor     eax, eax
lea     r12, service
lea     rbx, [rsp+0D8h+line]
lea     rbp, auth
mov     r14, [r13+0]
```

Durham University

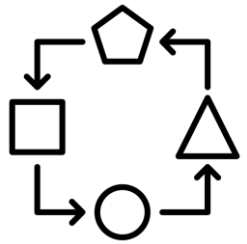
Input creation

Mutation-based

- Based on set of seed inputs
- Modifies existing input
- Especially effective when feedback is available

Generation-based

- Format of input is known beforehand
- Rules to derive inputs
- Often used when feedback not available



Durham University

Fuzzing Target: Binary vs Source

Source-based

- Allows for efficient instrumentation and sanitization
- Usually: Feedback collection via compile time instrumentation

Binary-based

- Instrumentation & Sanitization is challenging
- Solutions for feedback collection:
  - Tracing via hardware features
  - Emulation
  - Binary Rewriting



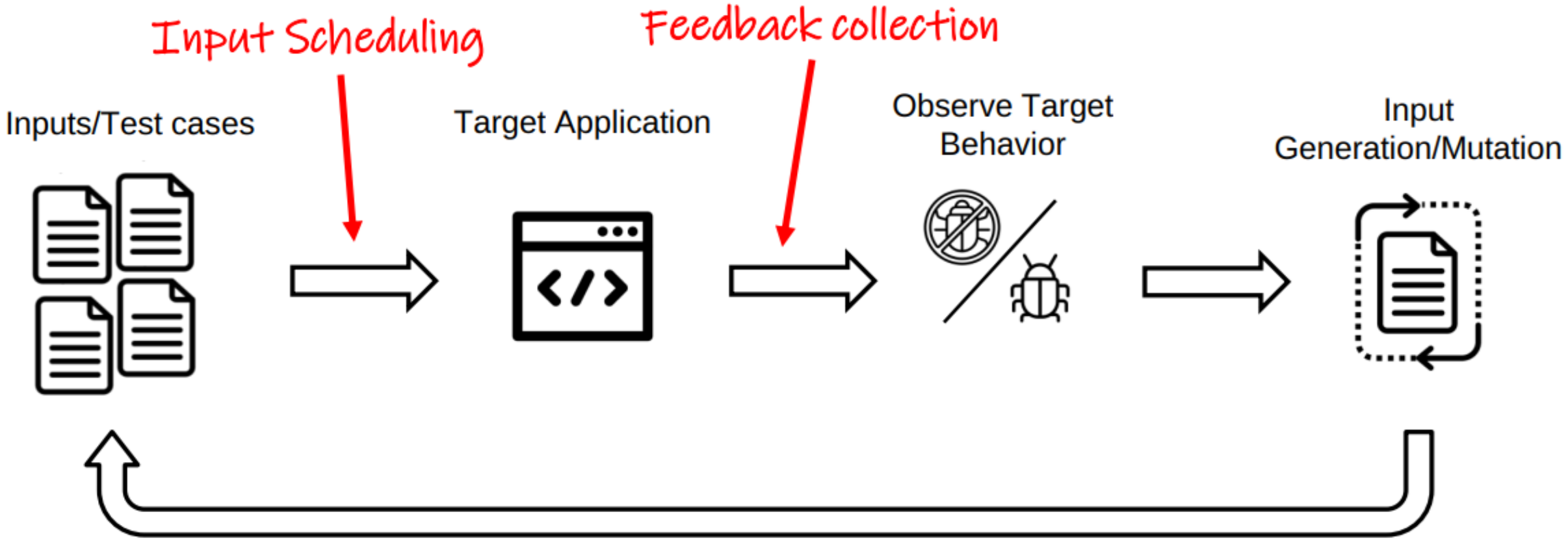
Durham University

Fuzzing: Additional Terminology

- Target: the software (or code unit) under test
- Harness: program to execute the target
- Corpus: A set of test inputs
- Seeds: The initial set of inputs given to the fuzzing process
- Queue: Test cases still to be executed
- Observation: Collected feedback of a test case execution



Fuzzing Algorithm



Fuzzing Algorithm

```
queue ← initSeedCorpus()
observations ← ∅
while ¬isDone(observations,queue) do
    candidate ← choose(queue, observations)
    mutated ← mutate(candidate,observations)
    observation ← eval(mutated)
    if isInteresting(observation,observations) then
        queue ← queue ∪ mutated
        observations ← observations ∪ observation
    end if
end while
```

Fuzzing algorithm (following definition of [1])

[1] Klees, G., Ruef, A., Cooper, B., Wei, S., & Hicks, M. “Evaluating fuzz testing”. *ACM CCS’18*

Fuzzing Algorithm

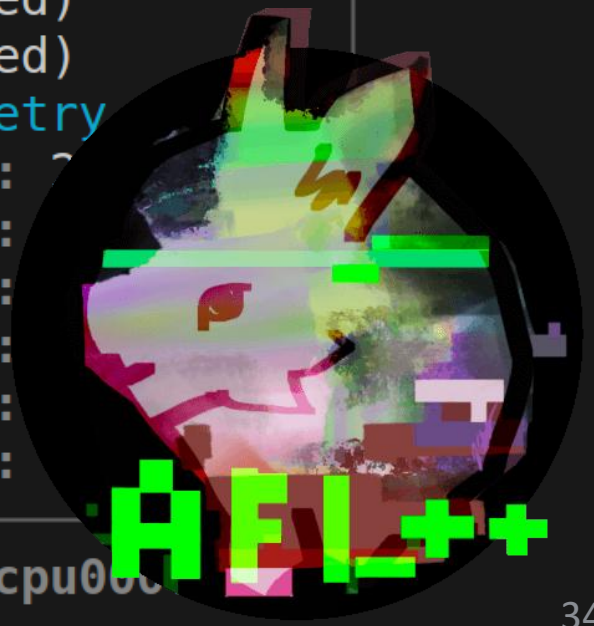
```
queue ← initSeedCorpus()
observations ← ∅
while ¬isDone(observations,queue) do
    candidate ← choose(queue, observations)
    mutated ← mutate(candidate,observations)
    observation ← eval(mutated)
    if isInteresting(observation,observations) then
        queue ← queue ∪ mutated
        observations ← observations ∪ observation
    end if
end while
```

Fuzzing algorithm (following definition by [1])

- **initSeedCorpus**: Initialize a new seed corpus.
- **isDone**: Determine if the fuzzing should stop or not based on progress toward a goal, or a timeout.
- **choose**: Choose at least one candidate seed from the queue for mutation.
- **mutate**: From at least one seed and any observations made about the program so far, produce a new candidate seed.
- **eval**: Evaluate a seed on the program to produce an observation.
- **isInteresting**: Determine if the observations produced from an evaluation on a mutated seed indicate that the input should be preserved or not.

[1] Klees, G., Ruef, A., Cooper, B., Wei, S., & Hicks, M. “Evaluating fuzz testing”. *ACM CCS’18*

```
american fuzzy lop ++4.08c {default} (python) [fast]
┌─ process timing ─────────────────────┬─ overall results ─┐
│        run time : 0 days, 0 hrs, 0 min, 27 sec │  cycles done : 0  │
│   last new find : 0 days, 0 hrs, 0 min, 3 sec  │ corpus count : 34 │
│last saved crash : none seen yet      │saved crashes : 0  │
│ last saved hang : none seen yet      │  saved hangs : 0  │
├─ cycle progress ─────┬─ map coverage ─┴──────────────────┘
│  now processing : 6.0 (17.6%)  │   map density : 0.19% / 0.53%
│  runs timed out : 0 (0.00%)    │count coverage : 1.56 bits/tuple
├─ stage progress ─────┼─ findings in depth ──────────────┐
│  now trying : havoc            │favored items : 9 (26.47%)
│ stage execs : 7240/12.8k (56.56%)│ new edges on : 18 (52.94%)
│ total execs : 7480             │total crashes : 0 (0 saved)
│  exec speed : 270.2/sec        │ total tmouts : 0 (0 saved)
├─ fuzzing strategy yields ─────┴─ item geometry ──┐
│   bit flips : disabled (default, enable with -D)  │   levels : 2
│  byte flips : disabled (default, enable with -D)  │  pending :
│ arithmetics : disabled (default, enable with -D)  │ pend fav :
│  known ints : disabled (default, enable with -D)  │own finds :
│  dictionary : n/a                                 │ imported :
│havoc/splice : 0/0, 0/0                           │stability :
│py/custom/rq : unused, unused, unused, unused      ├──────────┘
│    trim/eff : 0.00%/1, disabled                   │ [cpu00
├─ strategy: explore ──────────── state: started :-) ┘
```



AFL/AFL++

- Coverage-based, mutation-based fuzzer
  - Tremendous impact in industry and academia
- 3 main design principles:
  - Speed, reliability, ease of use [1]
  - Especially ease of use helped to success
- Software-suite rather than fuzzer alone
  - Tooling for instrumentation, human inspection, test minimization, and crash deduplication



Durham University

[1] https://lcamtuf.coredump.cx/afl/technical_details.txt; Image source: https://en.wikipedia.org/wiki/File:Conejillo_de_indias.jpg

AFL/AFL++: Core operation (simplified)

- afl-fuzz provides test case to target via stdin or files
  - Queue, crashes, hangs, and fuzzer stats are also all saved as files!
- Target executes testcase and tracks coverage via bitmap
  - Bitmap is analyzed by fuzzer after test case execution
  - If a test case generated new coverage, safes it in queue
- Input mutation via multiple different strategies
  - Repeatedly mutates and executes the same test file

The afl-fuzz Commandline

-i Where to read input seeds from
-o Where to place outputs



Environment variables control AFL's behaviour.
Almost everything in AFL++ is configurable by env variables.

Example:
`AFL_AUTORESUME`:
if restarted, resume fuzzing

the afl-fuzz command

timeout: 1 second per testcase

This is the main node (vs -S)

Memory limit

the target

@@:
Will be replaced with input file (by afl-fuzz)

Durham University

AFL/AFL++ Instrumentation

- By default, added during compile time
  - AFL-CC as “drop-in” compiler replacement for clang/gcc:
    - $ afl-cc target.c
  - However, options for binary fuzzing exist as well, e.g., qemu-mode or frida mode
  - Adds coverage tracking and forkserver logic

```
cur_location = <COMPILE_TIME_RANDOM>;
shared_mem[cur_location ^ prev_location]++;
prev_location = cur_location >> 1;
```

AFL/AFL++ Mutation Strategies

Variety of strategies, for instance:

- Trim: Reduce test case to shortest form yielding same coverage
- Arith: subtraction or addition to input values
- Bitflip: flip certain bits in the input
- Havoc: combination of multiple strategies, including block deletion, duplication, and dictionary additions (if applicable)
- Sync: In multi-process fuzzing setups, testcases from other fuzzers



Image source: https://lcamtuf.coredump.cx/afl/rabbit.gif

Minimization tools

afl-cmin:

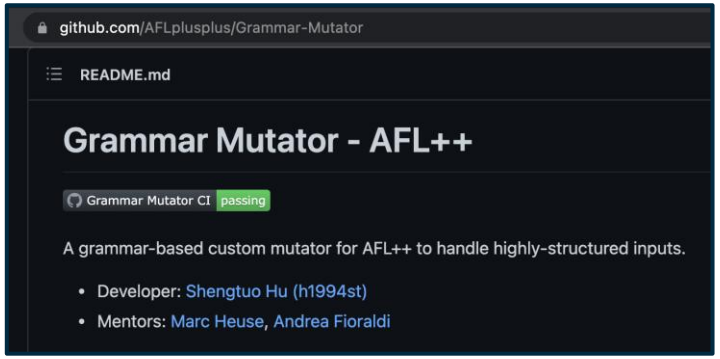
- Corpus Minimization
- Removes testcases that don't find new paths
- Smaller corpus ⇒ focus on the important testcases

afl-tmin:

- Minimizes a single testcase
- Tries to find the smallest possible input

Grammar based fuzzing with AFL++

- AFL++ can use Grammars for better input generation
- You can also write custom mutators from scratch




```
"<A>": [["<START_LINE>", "\r\n", "<HEADERS>", "<BODY>", "\r\n\r\n"]],

"<START_LINE>": [["<METHOD>", " ", "<URI>", " ", "<VERSION>"]],

"<METHOD>": [["GET"], ["HEAD"], ["POST"], ["PUT"], ["DELETE"], ["CONNECT"], ["OPTIONS"], ["TRACE"], ["PATCH"], ["ACL"], ["BASELINE-CONTROL"], ["BIND"],

"<URI>": [["<SCHEME>" , ":", "<HIER>", "<QUERY>", "<FRAGMENT>"]],

"<SCHEME>": [["http"], ["https"], ["shttp"], ["dav"], ["about"], ["attachment"], ["cid"], ["data"], ["file"], ["ftp"], ["ssh"], ["sip"]],
```

Sanitizers

- Bugs, even when triggered, do not necessarily result in a crash
  - Sanitizers aim to change this
- Example sanitizers:
  - Address Sanitizers (ASan): Memory corruptions
  - ThreadSanitizer (TSan): Race conditions, deadlocks
  - Memory Sanitizer (MSAN): Uninitialized memory
  - UBSan: Undefined behaviour
- (Probably) most relevant for fuzzing: ASAN
  - For gcc/clang: added at compile time via -fsanitize=address
  - Supported by `afl-cc` via `AFL_USE_ASAN=1` environment variable

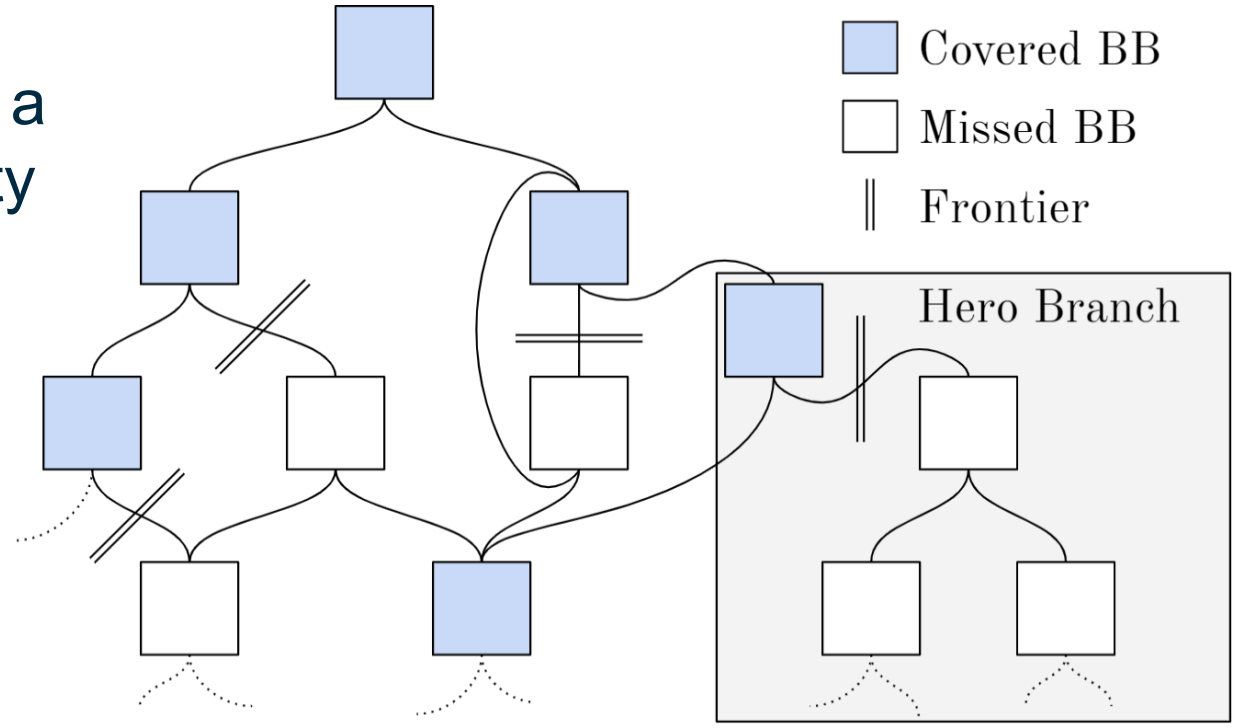
```
root@09f51fbc02b1:/vuln# gcc -Wno-stringop-overflow -fsanitize=address -o vuln vuln.c
root@09f51fbc02b1:/vuln# ./vuln
AAAA
```

Fuzzing Roadblocks/Coverage Frontiers

Sometimes, the fuzzer gets stuck at a branch that hides a lot of functionality

Can be addressed by:

- Harnessing
- Seed selection
- Tooling



Additional resources & further reading

- Fuzzing book:
  - https://www.fuzzingbook.org/
- Libafl documentation:
  - https://aflplus.plus/libafl-book/libafl.html
- AFL++ technical details:
  - https://blog.ritsec.club/posts/afl-under-hood/
- Additional perspective on AFL's design choices:
  - Fioraldi et al. "Dissecting American Fuzzy Lop A FuzzBench Evaluation." FUZZING'22
- Recent paper on evaluating Fuzzing:
  - Schloegel et al. "SoK: Prudent Evaluation Practices for Fuzzing." IEEE S&P'24.

Take home messages

- Software testing is a wide field, relevant outside security
- Fuzzing is a very effective tool for detecting hard-to-find issues
- But now AI also enhances static analysis
- Next lecture: Digital forensics

Durham University

Thanks! Questions & discussion